

HDOC DAY – 16

Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer. Define one separate user-defined function for each menu option, *with void return type* and *with a simple function call (without call by value/reference)*. Each function should read the required input, process it, and display the result. Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases.

- Check whether its digits are in increasing order from left to right.
- Check whether its digits are in decreasing order from left to right.
- Find the largest digit and the smallest digit.
- Find the second largest digit and second smallest digit in the number.

Sure. Here is a B.Tech CSE 1st-year level answer, using basic C concepts: switch-case, loops, if-else, %, /, and four separate void functions with no arguments.

Algorithm:

1. Start.
2. Display the menu:
 - Check digits in increasing order.
 - Check digits in decreasing order.
 - Find largest and smallest digit.
 - Find second largest and second smallest digit.
3. Input the user's choice.
4. Use switch-case.

5. Call the corresponding void function without passing any argument.
6. If the choice is invalid, display "Invalid choice".
7. Stop.

Function 1: Check Increasing Order

1. Input a positive integer.
2. Take the last digit using % 10.
3. Remove the last digit using / 10.
4. Compare adjacent digits from right to left.
5. If a left digit is greater than or equal to the digit on its right, the digits are not in strictly increasing order.
6. Otherwise, the digits are in increasing order.
7. Display the result.

Function 2: Check Decreasing Order

1. Input a positive integer.
2. Take the last digit.
3. Remove the last digit.
4. Compare adjacent digits from right to left.
5. If a left digit is less than or equal to the digit on its right, the digits are not in strictly decreasing order.
6. Otherwise, the digits are in decreasing order.

Function 3: Largest and Smallest Digit

1. Input a positive integer.
2. Extract each digit using % 10.
3. Initially set largest as 0 and smallest as 9.
4. Compare each digit with largest and smallest.
5. Update them whenever necessary.
6. Display the largest and smallest digits.

Function 4: Second Largest and Second Smallest Digit

1. Input a positive integer.
2. Use an array of size 10 to record which digits are present.
3. Extract every digit and mark it as present.
4. Search from 9 to 0 to find the largest and then the second distinct largest digit.
5. Search from 0 to 9 to find the smallest and then the second distinct smallest digit.
6. Display the results.
7. If fewer than two distinct digits exist, display that the second largest/smallest digit does not exist.

C PROGRAM:

```
#include <stdio.h>

/* Function declarations */

void checkIncreasing(void);
void checkDecreasing(void);
void findLargestSmallest(void);
void findSecondLargestSmallest(void);

int main()
{
    int choice;

    printf("----- MENU -----\\n");
    printf("1. Check digits in increasing order\\n");
    printf("2. Check digits in decreasing order\\n");
    printf("3. Find largest and smallest digit\\n");
    printf("4. Find second largest and second smallest digit\\n");
```

```
printf("Enter your choice: ");  
scanf("%d", &choice);  
  
switch (choice)  
{  
    case 1:  
        checkIncreasing();  
        break;  
  
    case 2:  
        checkDecreasing();  
        break;  
  
    case 3:  
        findLargestSmallest();  
        break;  
  
    case 4:  
        findSecondLargestSmallest();  
        break;  
  
    default:  
        printf("Invalid choice\n");  
}  
  
return 0;  
}
```

```
/* Function to check increasing order */  
void checkIncreasing(void)  
{  
    int num, current, previous;  
    int flag = 1;  
  
    printf("Enter a positive integer: ");  
    scanf("%d", &num);  
  
    previous = num % 10;  
    num = num / 10;  
  
    while (num > 0)  
    {  
        current = num % 10;  
  
        if (current >= previous)  
        {  
            flag = 0;  
            break;  
        }  
  
        previous = current;  
        num = num / 10;  
    }
```

```
if (flag == 1)
    printf("Digits are in increasing order\n");
else
    printf("Digits are not in increasing order\n");
}
```

```
/* Function to check decreasing order */
```

```
void checkDecreasing(void)
```

```
{
    int num, current, previous;
    int flag = 1;
```

```
    printf("Enter a positive integer: ");
    scanf("%d", &num);
```

```
    previous = num % 10;
    num = num / 10;
```

```
    while (num > 0)
    {
        current = num % 10;

        if (current <= previous)
        {
            flag = 0;
            break;
        }
    }
```

```
        previous = current;
        num = num / 10;
    }

    if (flag == 1)
        printf("Digits are in decreasing order\n");
    else
        printf("Digits are not in decreasing order\n");
}
```

/* Function to find largest and smallest digit */

```
void findLargestSmallest(void)
```

```
{
    int num, digit;
    int largest = 0;
    int smallest = 9;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &num);
```

```
    while (num > 0)
```

```
    {
        digit = num % 10;
```

```
        if (digit > largest)
```

```
            largest = digit;
```

```
        if (digit < smallest)
            smallest = digit;

        num = num / 10;
    }

    printf("Largest digit = %d\n", largest);
    printf("Smallest digit = %d\n", smallest);
}
```

```
/* Function to find second largest and second smallest digit */
```

```
void findSecondLargestSmallest(void)
```

```
{
    int num, digit, i;
    int present[10] = {0};
    int largest = -1, secondLargest = -1;
    int smallest = -1, secondSmallest = -1;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &num);
```

```
/* Record the digits present in the number */
```

```
while (num > 0)
```

```
{
    digit = num % 10;
    present[digit] = 1;
```

```

        num = num / 10;
    }

    /* Find largest and second largest */
    for (i = 9; i >= 0; i--)
    {
        if (present[i] == 1)
        {
            if (largest == -1)
                largest = i;
            else
            {
                secondLargest = i;
                break;
            }
        }
    }

    /* Find smallest and second smallest */
    for (i = 0; i <= 9; i++)
    {
        if (present[i] == 1)
        {
            if (smallest == -1)
                smallest = i;
            else
            {
                secondSmallest = i;

```

```

        break;
    }
}
}

if (secondLargest == -1)
    printf("Second largest digit does not exist\n");
else
    printf("Second largest digit = %d\n", secondLargest);

if (secondSmallest == -1)
    printf("Second smallest digit does not exist\n");
else
    printf("Second smallest digit = %d\n", secondSmallest);
}

```

Test case table:

Test case	choice	input	Expected output
1	1	12345	Digits are in increasing order
2	2	97351	Digits are not in decreasing order
3	3	58321	Largest=8, smallest=1
4	4	58321	Second largest=5, second smallest=2
5	5	--	Invalid choice

```
(kali㉿kali)-[~]
└─$ vi h16.c

(kali㉿kali)-[~]
└─$ gcc h16.c -o h16

(kali㉿kali)-[~]
└─$ ./h16
  MENU
1. Check digits in increasing order
2. Check digits in decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 1
Enter a positive integer: 12369
Digits are in increasing order

(kali㉿kali)-[~]
└─$ ./h16
  MENU
1. Check digits in increasing order
2. Check digits in decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 2
Enter a positive integer: 97351
Digits are not in decreasing order

(kali㉿kali)-[~]
└─$ ./h16
  MENU
1. Check digits in increasing order
2. Check digits in decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 58321
Invalid choice
```